

中断上下文

站在程序的角度看，中断随时可能发生，如果中断处理结束之后改变了寄存器的值，那么程序原先保存在寄存器里的值就会丢失；很明显，这会引入错误

处理器当然不能让上述的情况发生，

举个例子

```
# 主程序
#.....
mov r0, #0x01
mov r1, #0x02
svc #0x03
mov r3, #0x03
#.....

# SVC中断服务
SVC_Handler:
    mov r0, #0x05
    mov r1, #0x06
    bx lr

# 执行结果
# 从SVC_Handler跳转回主程序之后
# r0的值仍是0x01
# r1的值仍是0x02
```

所以，尽管中断改变了执行流程，但是并没有改变r0和r1寄存器的值

很明显，在执行SVC_Handler之前，有一些寄存器的值被保存了下来，执行完成之后，这些值又被覆写了回去，这个过程类似于程序调用利用栈保存寄存器的值，只不过对中断服务而言，这个过程是硬件完成的，不需要软件写什么代码。

Caller Saved Register 调用者保存寄存器

xPSR程序状态寄存器

return address 返回地址

LR

R12

R3、R2、R1、R0

这8个寄存器被成为“调用者保存寄存器”，按照Arm Architecture Procedure Call Standard (AAPCS arm 架构程序调用标准)，C编译器编写的函数会调用这8个寄存器，为了让C语言函数能成为中断服务，异常机制（中断机制）会在跳转到中断服务前保存这8个寄存器

中断发生的时候，这8个寄存器的内容会被压入栈中

xPSR程序状态寄存器

这个寄存器记录了当前程序的执行状态，他将32位的寄存器划分成3个部分记录不同信息：

APSR (31:24) —— 算术结果标志

- **N** (bit31): 负数
- **Z** (bit30): 零
- **C** (bit29): 进位/借位
- **V** (bit28): 溢出
- **Q** (bit27): 饱和 (DSP 指令)

EPSR (23:16) —— 执行状态

- **T** (bit24): Thumb 状态 (Cortex-M **必须=1**, 为 0 会 HardFault)
- **IT** (bits15:10 和 26:25): IF-THEN 条件执行块状态
- 23:16 还有部分 SIMD/DSP 状态 (M4/M7)

IPSR (15:0) —— 当前异常号

- 0 = 线程模式 (不在中断里)
- 1 = Reset
- 2 = NMI
- 3 = HardFault
- 11 = SVC
- 15 = SysTick
- 16+ = 外部 IRQ (IRQn = IPSR-16)

return address 返回地址

这不是一个寄存器，它代表了中断触发的时候，正在执行的程序原本要执行的下一条指令的地址

```
mov r0, #0x01
mov r1, #0x02
svc #0x03      # 这里触发中断
mov r3, #0x03 # return address记录的是这条指令的地址
```

至于为什么需要存一个return address？按照函数调用的习惯，直接把返回地址存在LR里不好吗？

当然不好，LR有自己的事情要做

LR链接寄存器

这里把LR再拿出来讲一下，是因为LR寄存器有一个之前一直没有提到的功能，LR也负责保存异常返回值

在要进入中断服务之前，原函数的LR被压入栈后，LR寄存器会保存当前程序的现场状态（用MSP还是PSP？是Handler状态还是Thread状态？），这个状态被一个叫做EXC_RETURN的状态码所描述

所以中断服务里最后的"bx LR"，处理器不会像常规函数返回一样，而是进行特殊处理：

处理器发现LR里是EXC_RETURN，会使用这个值恢复现场，然后再将之前压入栈的寄存器值恢复到寄存器里，最后根据return address跳转到原先的进程中继续执行

其余寄存器不做解释

SVC指令携带的参数要如何读取？

```
svc #0x03
```

这行代码除了触发SVC中断，还携带了一个立即数参数，如何在SVC_Handler中获取这个参数？

首先要了解一下SVC指令的机器码是什么样子，SVC的机器码占16bits，其中15：12表示条件码，11：8是固定的0111，表示这是一条SVC指令，7：0可以放一个长度位8bits的立即数

这是deepseek给出的获取立即数方案：

SVC_Handler:

```
TST    LR, #0x04      @ 判断使用 MSP(0) 还是 PSP(1)
MRSEQ  R0, MSP        @ 使用主栈
MRSNE  R0, PSP        @ 使用进程栈

LDR     R1, [R0, #24]  @ R1 = 栈中的 PC
SUB     R1, R1, #2     @ PC-2 = SVC 指令地址
LDRH    R2, [R1]       @ 读取 SVC 指令（16位）
AND     R0, R2, #0xFF  @ R0 = 立即数 0xA5

@ === 此时 R0 = 0xA5 ===
@ 你可以在这里根据 R0 分发 SVC

BX      LR
```

为什么TST LR, #0x04可以用于检查使用MSP还是PSP？

EXC_RETURN 中

0xFFFF FFF9表示使用MSP 1111 1111 1111 1111 1111 1111 1001

0xFFFF FFFD表示使用PSP 1111 1111 1111 1111 1111 1111 1101

所以取这个立即数的步骤有：

1. 判断现在用的是MSP还是PSP
2. 获取栈地址
3. 以栈地址倒推6个字节，找到return address的值
4. return address的地址值再 减去 2，得到svc #0x03这条命令在内存中的地址
5. 获取SVC这条指令的机器码，使用LDRH的原因是，SVC这条指令机器码，只占2个字节（16bits）
6. 通过AND操作获取想要的立即数

如果在终端服务中push但不pop会怎么样？

当然会出现严重的问题...因为恢复现场要依赖栈中的值，如果在中断服务中进行了push，但没有把push到栈里的值pop出来，恢复的时候使用的值就是我们自己push进去的值而非原来的上下文环境了